

A Project Report

On

PLANT DISEASE IDENTIFICATION

Submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

By

**AMARA. SUMANTH
(228H1A05C0)**

Under the Esteemed Guidance of

Mr. SATTU VENKATESWARA RAO, M.Tech

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**MVR College of Engineering & Technology
(AUTONOMOUS)**

(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008,
Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)
Paritala, NTR Dist., PIN 521180 A.P. India.

2025-2026

A Project Report

On

**AN ARTIFICIAL INTELLIGENCE AND CLOUD BASED COLLABORATIVE
PLATFORM FOR PLANT DISEASE IDENTIFICATION, TRACKING AND
FORECASTING FOR FARMERS**

Submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

By

**AMARA. SUMANTH
(228H1A05C0)**

Under the Esteemed Guidance of

Mr. SATTU VENKATESWARA RAO, M.Tech

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MVR College of Engineering & Technology

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008,

Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)

Paritala, NTR Dist., PIN 521180 A.P. India.

2025-2026

MVR College of Engineering & Technology

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008,
Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)
Paritala, NTR Dist., PIN 521180 A.P. India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE



This is to certify that the Project Report entitled “Project Title” is being submitted by “**AMARA. SUMANTH** (228H1A05C0)” in partial fulfillment for the award of the Degree of Bachelor of Technology in Department of COMPUTER SCIENCE AND ENGINEERING of MVR College of Engineering & Technology, for the record of Bonafide work carried out by them.

Mr. SATTU VENKATESWARA RAO
Assistant Professor
Project Supervisor

Mr. CH. SREENU BABU
Associate Professor
Head of the Department

Examiner

ACKNOWLEDGEMENT

I express my deep sense of gratitude and indebtedness to the **Management & Dr.U. YEDUKONDALU, Principal** of our college, who gave very good support to me during this work and others who assisted me in this project work.

I also thank **Mr.CH. SREENU BABU**, Head of the Department of Computer Science And Engineering, who gave very good support to me during this work.

I also thank my **Project Supervisor, Mr. SATTU VENKATESWARA RAO, Asst. Professor of Computer Science And Engineering Department, MVR College of Engineering & Technology**, for her valuable guidance during the course of this project work. I am much indebted to her for suggesting a challenging and interactive project and her valuable advice at every stage of this work. I am very much thankful to her for her coordination in this regard.

Similarly, I am grateful to my friends I have worked with and consider myself extremely privileged and fortunate to work with such honest, hard working and loyal friends.

I wish my warm and grateful thanks to the staff of our department of **MVR College of Engineering & Technology** for the assistance for their continuous encouragement in completing my work successfully.

Finally I thank one and all who directly or indirectly helped me in completing project successfully.

AMARA. SUMANTH
(228H1A05C0)

DECLARATION

I hereby declare that this project entitled **“AN ARTIFICIAL INTELLIGENCE AND CLOUD BASED COLLABORATIVE PLATFORM FOR PLANT DISEASE IDENTIFICATION, TRACKING AND FORECASTING”** has been undertaken by me and this work has been submitted to **MVR college of Engineering & Technology affiliated to JNTUK, Kakinada**, in partial fulfillment of the requirements for award of the degree of **Bachelor of Technology in Department of Computer Science And Engineering**.

I further declare that this project work has not been submitted in full or part for the award of any degree of this in any other educational institutions.

**AMARA. SUMANTH
(228H1A05C0)**

ABSTRACT

Plant diseases are a major threat to farmers, consumers, environment and the global economy. In India alone, 35% of field crops are lost to pathogens and pests causing losses to farmers. Indiscriminate use of pesticides is also a serious health concern as many are toxic and biomagnified. These adverse effects can be avoided by early disease detection, crop surveillance and targeted treatments. Most diseases are diagnosed by agricultural experts by examining external symptoms. However, farmers have limited access to experts. This project helps in creation of platform for automated disease diagnosis tracking and forecasting. Farmers can instantly and accurately identify diseases and get solutions with a mobile app by photographing affected plant parts. Real-time diagnosis is enabled using the latest Artificial Intelligence (AI) algorithms for Cloud-based image processing. The AI model continuously learns from user uploaded images and expert suggestions to enhance its accuracy. Farmers can also interact with local experts through the platform. For preventive measures, disease density maps with spread forecasting are rendered from a Cloud based repository of geotagged images and micro-climactic factors. A web interface allows experts to perform disease analytics with geographical visualizations. In our experiments, the AI model (CNN) was trained with large disease datasets, created with plant images self-collected from many farms over 7 months. Test images were diagnosed using the automated CNN model and the results were validated by plant pathologists. Over 95% disease identification accuracy was achieved. Our solution is a novel, scalable and accessible tool for disease management of diverse agricultural crop plants and can be deployed as a Cloud based service for farmers and experts for ecologically sustainable crop production.

INDEX

Chapter / Section	Description	Page No.
CHAPTER 1	INTRODUCTION	1
1.1	Introduction	1
CHAPTER 2	LITERATURE SURVEY	3
CHAPTER 3	SYSTEM ANALYSIS	6
3.1	Existing System	6
3.2	Propose System	6
3.3	Feasibility Study	7
3.4	Input Design	8
3.5	Output Design	9
CHAPTER 4	SYSTEM REQUIREMENTS SPECIFICATION	11
4.1	Functional Requirements	11
4.2	Non Functional Requirements	12
CHAPTER 5	SYSTEM DESIGN	16
5.1	System Specifications	16
5.2	System Architecture	16
5.3	UML Diagrams	17
5.3.1	Use Case Diagram	18
5.3.2	Class Diagram	19
5.3.3	Sequence Diagram	20
5.3.4	Activity Diagram	21
5.3.5	Collaboration Diagram	22
CHAPTER 6	SYSTEM IMPLEMENTATION	24
6.1	Modules Description	24
6.2	Sample Code	25
6.3	System Testing	34
6.4	Types of Testing	34

Chapter / Section	Description	Page No.
6.5	Testing Methodologies	37
CHAPTER 7	SCREENSHOTS	42
CHAPTER 8	CONCLUSION	48
8.1	Future Work and Extensions	48
APPENDIX-A	BIBLIOGRAPHY	50
APPENDIX-B	TECHNOLOGY USED	53

LIST OF FIGURES

Figure No	Figure Name	Figure Name
Fig 5.2	System Architecture	17
Fig 5.3.1	Use Case Diagram	19
Fig 5.3.2	Class Diagram	20
Fig 5.3.3	Sequence Diagram	21
Fig 5.3.4	Activity Diagram	22
Fig 5.3.5	Collaboration Diagram	23

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION

Agriculture is fundamental to human survival. For populated developing countries like India, it is even more imperative to increase the productivity of crops, fruits and vegetables. Not only productivity, the quality of produce needs to stay high for better public health. However, both productivity and quality of food gets hampered by factors such as spread of diseases that could have been prevented with early diagnosis. Many of these diseases are infectious leading to total loss of crop yield. Given the vast geographical spread of agricultural lands, low education levels of farmers coupled with limited awareness and lack of access to plant pathologists, human assisted disease diagnosis is not effective and cannot keep up with the exorbitant requirements. To overcome the shortfall of human assisted disease diagnosis, it is imperative to build automation around crop disease diagnosis with technology and introduce low cost and accurate machine assisted diagnosis easily accessible to farmers. Some strides have been made in applying technologies such as robotics and computer vision systems to solve myriad problems in the agricultural domain. The potential of image processing has been explored to assist with precision agriculture practices, weed and herbicide technologies, monitoring plant growth and plant nutrition management [1][2]. However, progress on automating plant disease diagnosis is still rudimentary in spite of the fact that many plant diseases can be identified by plant pathologists by visual inspection of physical symptoms such as detectable change in color, wilting, appearance of spots and lesions etc. along with soil and climatic conditions. Overall, the commercial level of investment in bridging agriculture and technology remains lower as compared to investments done in more lucrative fields such as human health and education. Promising research efforts have not been able to productize due to challenges such as access and linkage for farmers to plant pathologists, high cost of deployment and scalability of solution. Recent developments in the fields of Mobile technology, Cloud computing and Artificial Intelligence (AI) create a perfect opportunity for creating a scalable low-cost solution for crop diseases that can be widely deployed. In developing countries such as India, mobile phones with internet connectivity have become ubiquitous. Camera and GPS enabled low cost mobile phones are widely available that can be leveraged by individuals to upload images with geolocation. Over widely available mobile networks, they can communicate with more sophisticated Cloud based backend services which can perform the compute heavy tasks, maintain a centralized database, and perform data analytics. Another leap of technology in

recent years is AI based image analysis which has surpassed human eye capabilities and can accurately identify and classify images. The underlying AI algorithms use Neural Networks (NN) which have layers of neurons with a connectivity pattern inspired by the visual cortex. These networks get “trained” on a large set of pre-classified “labeled” images to achieve high accuracy of image classification on new unseen images. Since 2012 with “AlexNet” winning the ImageNet competition, deep Convolutional Neural Networks (CNNs) have consistently been the winning architecture for computer vision and image analysis [3]. The breakthrough in the capabilities of CNNs have come with a combination of improved compute capabilities, large data sets of images available and improved NN algorithms. Besides accuracy, AI has evolved and become more affordable and accessible with open source platforms such as TensorFlow [4]. Prior art related to our project includes initiatives to gather healthy and diseased crop images [5], image analysis using feature extraction [6], RGB images [7], spectral patterns [8] and fluorescence imaging spectroscopy [9]. Neural Networks have been used in the past for plant disease identification but the approach was to identify texture features. Our proposal takes advantage of the evolution of Mobile, Cloud and AI to develop an end-to-end crop diagnosis solution that simulates the expertise (“intelligence”) of plant pathologists and brings it to farmers. It also enables a collaborative approach towards continually increasing the disease database and seeking expert advice when needed for improved NN classification accuracy and tracking for outbreaks.

CHAPTER-2

LITERATURE SURVEY

2.1 A survey of image processing techniques for agriculture

AUTHORS: Lalit P. Saxena and Leisa J. Armstrong

ABSTRACT: Computer technologies have been shown to improve agricultural productivity in a number of ways. One technique which is emerging as a useful tool is image processing. This paper presents a short survey on using image processing techniques to assist researchers and farmers to improve agricultural practices. Image processing has been used to assist with precision agriculture practices, weed and herbicide technologies, monitoring plant growth and plant nutrition management. This paper highlights the future potential for image processing for different agricultural industry contexts.

2.2 Imagenet classification with deep convolutional neural networks

AUTHORS: A. Krizhevsky, I. Sutskever and G. E. Hinton,

ABSTRACT: We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

2.3 Integrating soms and a bayesian classifier for segmenting diseased plants in uncontrolled environments

AUTHORS: D. L. Hernández-Rabadán, F. Ramos-Quintana and J. Guerrero Juk

ABSTRACT: This work presents a methodology that integrates a nonsupervised learning approach (self-organizing map (SOM)) and a supervised one (a Bayesian classifier) for segmenting diseased plants that grow in uncontrolled environments such as greenhouses, wherein the lack of control of illumination and presence of background bring about serious drawbacks. During the training phase two SOMs are used: one that creates color groups of images, which are classified into two groups using K -means and labeled as vegetation and nonvegetation by using rules, and a second SOM that corrects classification errors made by the first SOM. Two color histograms are generated from the two color classes and used to estimate the conditional probabilities of the Bayesian classifier. During the testing phase an input image is segmented by the Bayesian classifier and then it is converted into a binary image, wherein contours are extracted and analyzed to recover diseased areas that were incorrectly classified as nonvegetation. The experimental results using the proposed methodology showed better performance than two of the most used color index methods.

2.4 Visible-near infrared spectroscopy for detection of Huanglongbing in citrus orchards

AUTHORS: S. Sankaran, A. Mishra, J. M. Maja and R. Ehsani

ABSTRACT: This paper evaluates the feasibility of applying visible-near infrared spectroscopy for in-field detection of Huanglongbing (HLB) in citrus orchards. Spectral reflectance data from the wavelength range of 350–2500nm with 989 spectral features were collected from 100 healthy and 93 HLB-infected citrus trees using a visible-near infrared spectroradiometer. During data preprocessing, the spectral data were normalized and averaged every 25nm to reduce the spectral features from 989 to 86. Three datasets were generated from the preprocessed raw data: first derivatives, second derivatives, and a combined dataset (generated by integrating preprocessed raw data, first derivatives and second derivatives). The preprocessed datasets were analyzed using principal component analysis (PCA) to further reduce the number of features used as inputs in the classification algorithm. The dataset consisting of principal components were randomized and separated into training and testing datasets such that 75% of the dataset was used for training; while 25% of the dataset was used for testing the classification algorithms. The number of samples in the training and testing datasets was 145 and 48, respectively. The classification algorithms tested were: linear

discriminant analysis, quadratic discriminant analysis (QDA), k-nearest neighbor, and soft independent modeling of classification analogies (SIMCA). The reported classification accuracies of the algorithms are an average of three runs. When the second derivatives dataset were analyzed, the QDA-based classification algorithm yielded the highest overall average classification accuracies of about 95%, with HLB-class classification accuracies of about 98%. In the combined dataset, SIMCA-based algorithms resulted in high overall classification accuracies of about 92% with low false negatives (less than 3%).

2.5 Rethinking the inception architecture for computer vision

AUTHORS: Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, Zbigniew Wojna

ABSTRACT: Convolutional networks are at the core of most state-of-the-art computer vision solutions for a wide variety of tasks. Since 2014 very deep convolutional networks started to become mainstream, yielding substantial gains in various benchmarks. Although increased model size and computational cost tend to translate to immediate quality gains for most tasks (as long as enough labeled data is provided for training), computational efficiency and low parameter count are still enabling factors for various use cases such as mobile vision and big-data scenarios. Here we explore ways to scale up networks in ways that aim at utilizing the added computation as efficiently as possible by suitably factorized convolutions and aggressive regularization. We benchmark our methods on the ILSVRC 2012 classification challenge validation set demonstrate substantial gains over the state of the art: 21.2% top-1 and 5.6% top-5 error for single frame evaluation using a network with a computational cost of 5 billion multiply-adds per inference and with using less than 25 million parameters. With an ensemble of 4 models and multi-crop evaluation, we report 3.5% top-5 error on the validation set (3.6% error on the test set) and 17.3% top-1 error on the validation set.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

- In India alone, 35% of field crops are lost to pathogens and pests causing losses to farmers. Indiscriminate use of pesticides is also a serious health concern as many are toxic and biomagnified. These adverse effects can be avoided by early disease detection, crop surveillance and targeted treatments. Most diseases are diagnosed by agricultural experts by examining external symptoms. However, farmers have limited access to experts.

DISADVANTAGES OF EXISTING SYSTEM:

- Indiscriminate use of pesticides is also a serious health concern as many are toxic and biomagnified.

3.2 PROPOSED SYSTEM

- Proposed system use convolution neural network as artificial intelligence to train all plant diseases images and then upon uploading new images CNN will predict plant disease available in uploaded images. For storing CNN train model and images author is using cloud services. so, using AI author predicting plant disease and cloud is used to store data.
- Proposed system use smart phone to upload image but designing android application will take extra cost and time so a python web application was built. Using this web application CNN model will get trained and user can upload images and then application will apply CNN model on uploaded images to predict diseases. If this web application deployed on real web server then it will extract users location from request object and can display those location in map.

ADVANTAGES OF PROPOSED SYSTEM:

- Accurately identify diseases and get solutions with a mobile app by photographing affected plant parts.

3.3 FEASIBILITY STUDY

Preliminary investigation examine project feasibility, the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All system is feasible if they are unlimited resources and infinite time. There are aspects in the feasibility study portion of the preliminary investigation:

- Economical Feasibility
- Operational Feasibility
- Technical Feasibility

ECONOMIC FEASIBILITY

A system can be developed technically and that will be used if installed must still be a good investment for the organization. In the economical feasibility, the development cost in creating the system is evaluated against the ultimate benefit derived from the new systems. Financial benefits must equal or exceed the costs.

The system is economically feasible. It does not require any addition hardware or software. Since the interface for this system is developed using the existing resources and technologies available at NIC, There is nominal expenditure and economical feasibility for certain.

OPERATIONAL FEASIBILITY

Proposed projects are beneficial only if they can be turned out into information system. That will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an important part of the project implementation. Some of the important issues raised are to test the operational feasibility of a project includes the following: -

- Is there sufficient support for the management from the users?
- Will the system be used and work properly if it is being developed and implemented?
- Will there be any resistance from the user that will undermine the possible application benefits?

This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration. So there is no question of resistance from the users that can undermine the possible application benefits.

The well-planned design would ensure the optimal utilization of the computer resources and would help in the improvement of performance status.

TECHNICAL FEASIBILITY

The technical issue usually raised during the feasibility stage of the investigation includes the following:

- Does the necessary technology exist to do what is suggested?
- Do the proposed equipments have the technical capacity to hold the data required to use the new system?
- Will the proposed system provide adequate response to inquiries, regardless of the number or location of users?
- Can the system be upgraded if developed?
- Are there technical guarantees of accuracy, reliability, ease of access and data security?

Earlier no system existed to cater to the needs of ‘Secure Infrastructure Implementation System’. The current system developed is technically feasible. It is a web based user interface for audit workflow at NIC-CSD. Thus it provides an easy access to the users. The database’s purpose is to create, establish and maintain a workflow among various entities in order to facilitate all concerned users in their various capacities or roles. Permission to the users would be granted based on the roles specified. Therefore, it provides the technical guarantee of accuracy, reliability and security. The software and hardware requirements for the development of this project are not many and are already available in-house at NIC or are available as free as open source. The work for the project is done with the current equipment and existing software technology. Necessary bandwidth exists for providing a fast feedback to the users irrespective of the number of users using the system.

3.4 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES:

1. To design a computer-based system which avoids errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.
2. To design user-friendly screens to enter the large volume of the data which is free from errors performs data manipulation.
3. To check and validate the screens and provide appropriate messages to user and create an input layout which is easy to follow.

3.5 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. To design a system which can use easily and effectively and generate the specific output that is needed to meet the requirements.
2. Select methods for presenting information.
3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the □ Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.

- Confirm an action.

CHAPTER-4

SYSTEM REQUIREMENTS SPECIFICATION

4.1 FUNCTIONAL REQUIREMENTS

In software engineering and systems engineering, a **functional requirement** defines a function of a system or its component, where a function is described as a specification of behavior between outputs and inputs.^[1]

Functional requirements may involve calculations, technical details, data manipulation and processing, and other specific functionality that define what a system is supposed to accomplish.^[2] Behavioral requirements describe all the cases where the system uses the functional requirements, these are captured in use cases. Functional requirements are supported by non-functional requirements (also known as "quality requirements"), which impose constraints on the design or implementation (such as performance requirements, security, or reliability). Generally, functional requirements are expressed in the form "system must do <requirement>," while non-functional requirements take the form "system shall be <requirement>." The plan for implementing functional requirements is detailed in the system design, whereas *non-functional* requirements are detailed in the system architecture.^{[4][5]}

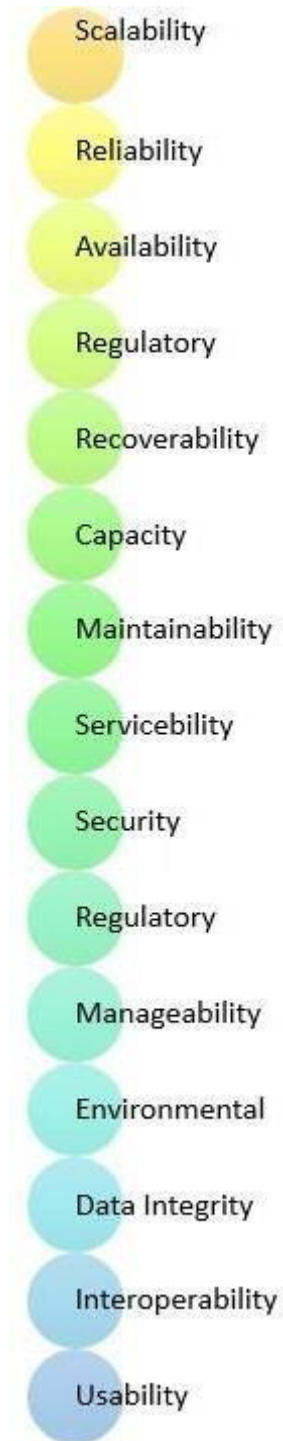
As defined in requirements engineering, functional requirements specify particular results of a system. This should be contrasted with non-functional requirements, which specify overall characteristics such as cost and reliability. Functional requirements drive the application architecture of a system, while non-functional requirements drive the technical architecture of a system.^[4]

In some cases a requirements analyst generates use cases after gathering and validating a set of functional requirements. The hierarchy of functional requirements collection and change, broadly speaking, is: user/stakeholder request → analyze → use case → incorporate. Stakeholders make a request; systems engineers attempt to discuss, observe, and understand the aspects of the requirement; use cases, entity relationship diagrams, and other models are built to validate the requirement; and, if documented and approved, the requirement is implemented/incorporated.^[6] Each use case illustrates behavioral scenarios through one or more functional requirements. Often, though, an analyst will begin by eliciting a set of use cases, from which the analyst can derive the functional requirements that must be implemented to allow a user to perform each use case.

4.2 NON-FUNCTIONAL REQUIREMENT

NON-FUNCTIONAL REQUIREMENT (NFR) specifies the quality attribute of a software system. They judge the software system based on Responsiveness, Usability, Security, Portability and other non-functional standards that are critical to the success of the software system. Example of nonfunctional requirement, *“how fast does the website load?”* Failing to meet non-functional requirements can result in systems that fail to satisfy user needs.

Non-functional Requirements allows to impose constraints or restrictions on the design of the system across the various agile backlogs. Example, the site should load in 3 seconds when the number of simultaneous users are > 10000. Description of non-functional requirements is just as critical as a functional requirement.



- Usability requirement
- Serviceability requirement
- Manageability requirement
- Recoverability requirement
- Security requirement
- Data Integrity requirement □ Capacity requirement
- Availability requirement

- Scalability requirement
- Interoperability requirement
- Reliability requirement
- Maintainability requirement
- Regulatory requirement
- Environmental requirement

Advantages of Non-Functional Requirement Benefits/pros

of Non-functional testing are:

- The nonfunctional requirements ensure the software system follow legal and compliance rules.
- They ensure the reliability, availability, and performance of the software system □ They ensure good user experience and ease of operating the software.
- They help in formulating security policy of the software system.

Disadvantages of Non-functional requirement

Cons/drawbacks of Non-function requirement are:

- None functional requirement may affect the various high-level software subsystem
- They require special consideration during the software architecture/high-level design phase which increases costs.
- Their implementation does not usually map to the specific software sub-system, □ It is tough to modify non-functional once you pass the architecture phase.

KEY LEARNING

- A non-functional requirement defines the performance attribute of a software system.
- Types of Non-functional requirement are Scalability Capacity, Availability, Reliability, Recoverability, Data Integrity, etc.
- Example of Non Functional Requirement is Employees never allowed to update their salary information. Such attempt should be reported to the security administrator.
- Functional Requirement is a verb while Non-Functional Requirement is an attribute

- The advantage of Non-functional requirement is that it helps you to ensure good user experience and ease of operating the software

The biggest disadvantage of Non-functional requirement is that it may affect the various high level software subsystems.

CHAPTER-5

SYSTEM DESIGN

5.1 SYSTEM SPECIFICATIONS

Hardware Requirements:

- System : Pentium IV 2.4 GHz.
- Hard Disk : 40 GB.
- Floppy Drive : 1.44 Mb.
- Monitor : 15 VGA Colour.
- Mouse : Logitech.
- Ram : 512 Mb.

Software Requirements:

- Operating system : Windows 8/10.
- Coding Language : PYTHON

5.2 SYSTEM ARCHITECTURE

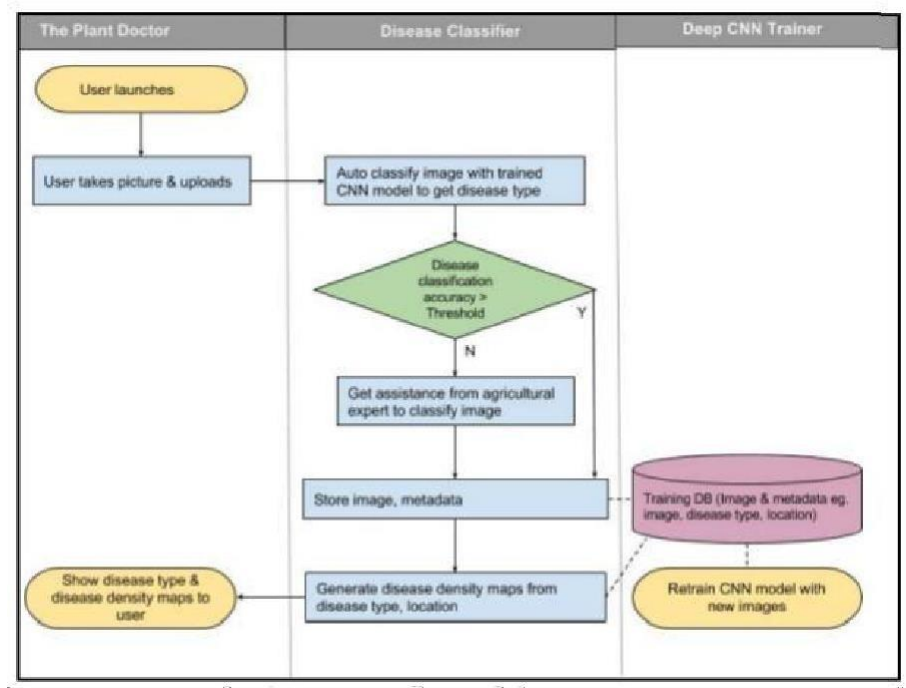


Fig 5.2 System Architecture

5.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful

In the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

5.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

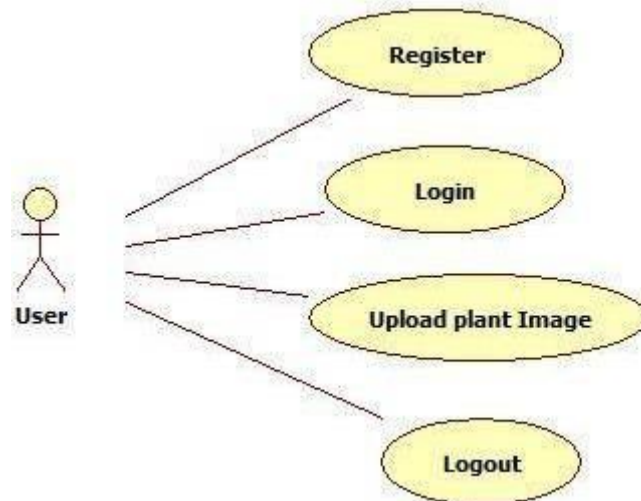


Fig 5.2.1 Use Case Diagram

5.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

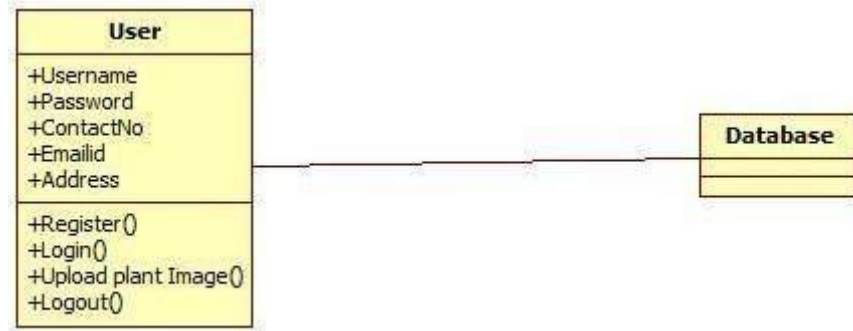


Fig 5.2.2 Class Diagram

5.2.3 SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

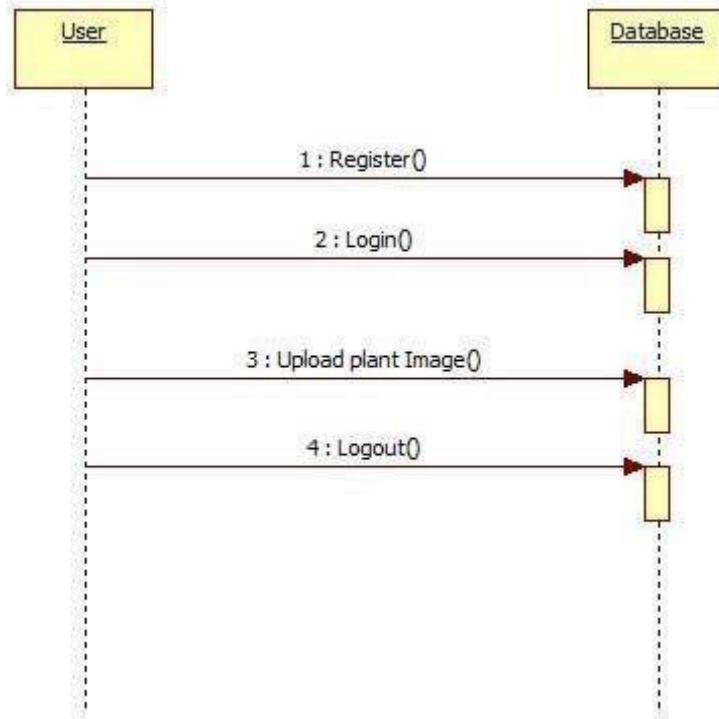


Fig 5.2.3 Sequence Diagram

5.2.4 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

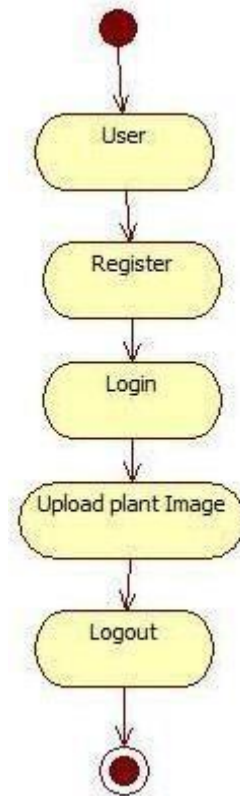


Fig 5.2.4 Activity Diagram

5.2.5 COLLABORATION DIAGRAM

The collaboration diagram is used to show the relationship between the objects in a system. Both the sequence and the collaboration diagrams represent the same information but differently. Instead of showing the flow of messages, it depicts the architecture of the object residing in the system as it is based on object-oriented programming. An object consists of several features. Multiple objects present in the system are connected to each other. The collaboration diagram, which is also known as a communication diagram, is used to portray the object's architecture in the system.



Fig 5.2.5 Collaboration Diagram

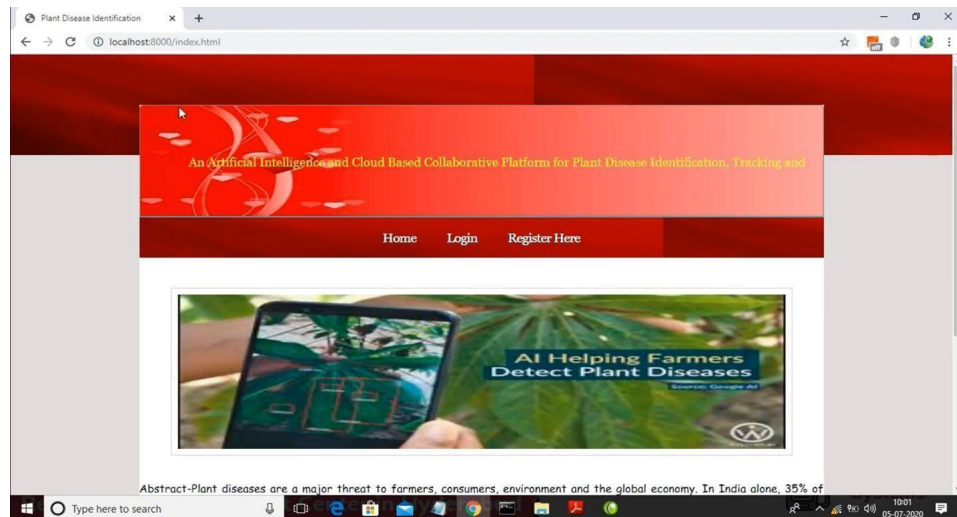
CHAPTER-6

SYSTEM IMPLEMENTATION

6.1 MODULES DESCRIPTION

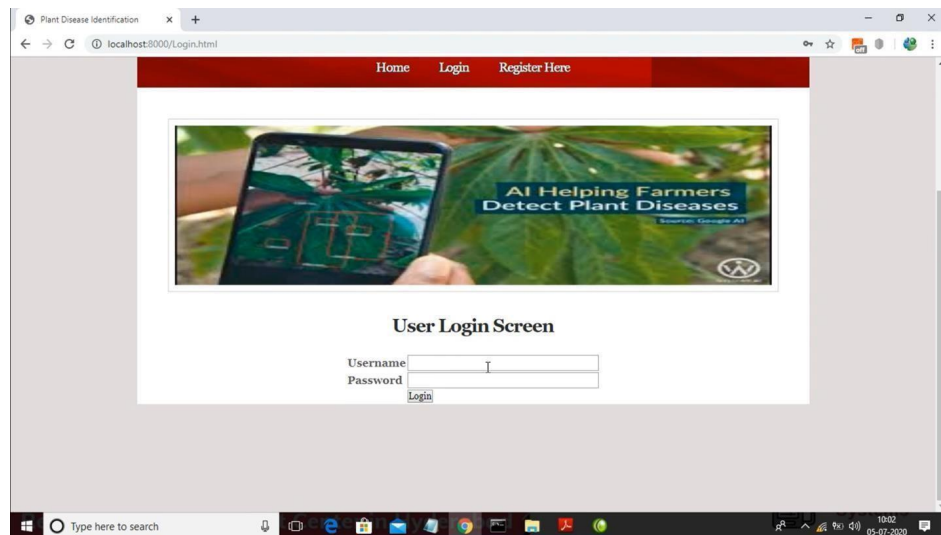
- **REGISTER :**

In this module user/former has to register himself.



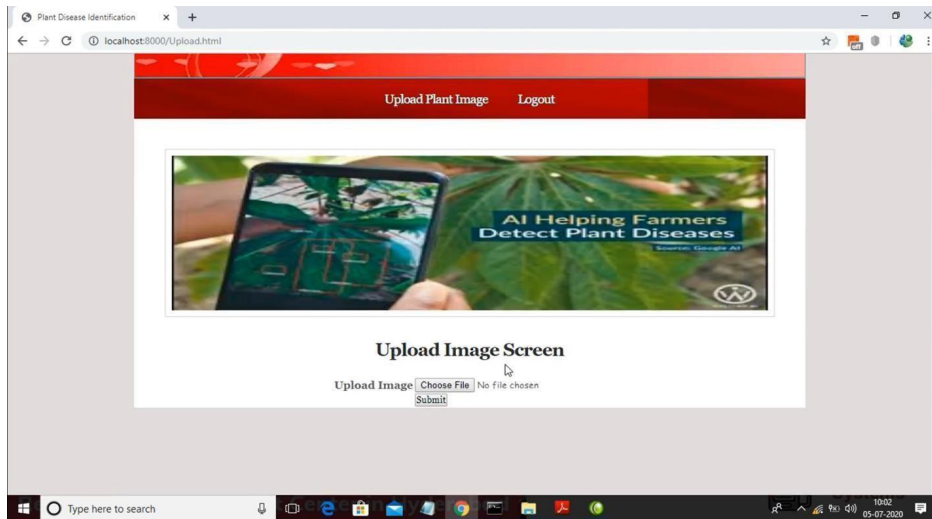
- **LOGIN:**

In this module user/former has to login with valid user name and password.



- **UPLOAD PLANT IMAGE:**

In this module user/former should upload plant image and can identify the plant disease.



- **LOGOUT:**

After completion of user activities can logout from the application by using this module.

6.2 SAMPLE CODE

PYTHON :-

```
from django.db import models
```

```
class PlantRecord(models.Model):
    username = models.CharField(max_length=100)
    image_name = models.CharField(max_length=255)
    disease = models.CharField(max_length=100)
    latitude = models.FloatField()
    longitude = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)
```

```
import cv2
import numpy as np
import tensorflow as tf
from django.shortcuts import render
from django.core.files.storage import FileSystemStorage
from .models import PlantRecord
```

```
# Global list of classes (truncated for brevity)
LABELS = ['Pepper_Bacterial_spot', 'Potato_Early_blight', 'Tomato_Healthy',
'Tomato_Late_blight']
```

```

# Load model once at startup
MODEL_PATH = 'model/vgg_model.json'
WEIGHTS_PATH = 'model/vgg_weights.h5'

def load_plant_model():
    with open(MODEL_PATH, "r") as f:
        model = tf.keras.models.model_from_json(f.read())
        model.load_weights(WEIGHTS_PATH)

    return model

classifier = load_plant_model()
def UploadImage(request):
    if request.method == 'POST' and request.FILES['t1']:

        # 1. Save File
        file = request.FILES['t1']
        fs = FileSystemStorage(location='static/plant/')
        filename = fs.save('test.png', file)

        # 2. Preprocess Image
        img = cv2.imread(f'static/plant/{filename}')
        img = cv2.resize(img, (64, 64))
        img_array = np.array(img).reshape(1, 64, 64, 3) / 255.0

        # 3. Predict
        preds = classifier.predict(img_array)
        prediction_idx = np.argmax(preds)
        result = LABELS[prediction_idx]

        # 4. Save to Database (Django ORM)
        PlantRecord.objects.create
        (username=request.session.get('username', 'Guest'),
         image_name=file.name,
         disease=result,
         latitude=17.38, # Default or from GeoIP
         longitude=78.45)

    return render(request, 'Map.html', {'prediction': result})

    return render(request, 'Upload.html')

```

BACKEND:-

```
from flask import Flask, request, jsonify
import os
from werkzeug.utils import secure_filename
import random

app = Flask(__name__)

UPLOAD_FOLDER = "uploads"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

# Create uploads folder
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

# Dummy disease prediction (replace with real AI model later)
def predict_disease(image_path):
    diseases = ["Leaf Spot", "Powdery Mildew", "Healthy", "Blight"]

    return random.choice(diseases)

@app.route("/predict", methods=["POST"])
def predict():
    if "file" not in request.files:
        return jsonify({"error": "No file uploaded"}), 400

    file = request.files["file"]

    if file.filename == "":
        return jsonify({"error": "Empty file"}), 400

    filename = secure_filename(file.filename)
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)
    result = predict_disease(filepath)
    return jsonify({
        "disease": result,
        "message": f"Detected: {result}"
    })

if __name__ == "__main__":
    app.run(debug=True)
```

FORNTEND:-

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [file, setFile] = useState(null);
  const [result, setResult] = useState("");

  const handleFileChange = (e) => {
    setFile(e.target.files[0]);
  };

  const handleUpload = async () => {
    if (!file) {
      alert("Please select an image");
      return;
    }

    const formData = new FormData();
    formData.append("file", file);

    try {
      const res = await axios.post("http://127.0.0.1:5000/predict", formData);
      setResult(res.data.disease);
    } catch (err) {
      console.error(err);
      alert("Error uploading file");
    }
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>☐ Plant Disease Detection</h2>

      <input type="file" onChange={handleFileChange} />
      <br /><br />

      <button onClick={handleUpload}>Upload & Predict</button>

      <h3>Result: {result}</h3>
    </div>
  );
}

export default App;
```

AI MODEL INTEGRATION:-

```
from tensorflow.keras.models import load_model
import numpy as np
from tensorflow.keras.preprocessing import image

model = load_model("model.h5")

def predict_disease(img_path):
    img = image.load_img(img_path, target_size=(224, 224))
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)
    classes = ["Leaf Spot", "Mildew", "Healthy", "Blight"]

    return classes[np.argmax(prediction)]
```

6.2 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.3 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests

at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : Identified classes of valid input must be accepted.

Invalid Input : Identified classes of invalid input must be rejected

Functions : Identified functions must be exercised.

Output : Identified classes of application outputs must be exercised.

Systems/Procedures: Interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. It is not possible to see inside it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6.4 TESTING METHODOLOGIES

The following are the Testing Methodologies:

- **Unit Testing.** ○ **Integration Testing.**
- **User Acceptance Testing.** ○ **Output Testing.**
- **Validation Testing.**

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing.

During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

The following are the types of Integration Testing:

1) Top Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner.

In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

2. Bottom-up Integration

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters that perform a specific Software sub-function.

- A driver (ie: the control program) for testing is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program Structure

The bottom up approaches tests each module individually and then each module is integrated with a main module and tested for functionality.

User Acceptance Testing

User Acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required. The system developed provides a friendly user interface that can easily be understood even by a person who is new to the system.

Output Testing

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specified format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration. Hence the output format is considered in 2 ways – one is on screen and another in printed format.

Validation Checking

Validation checks are performed on the following fields.

Text Field:

The text field can contain only the number of characters lesser than or equal to its size. The text fields are alphanumeric in some tables and alphabetic in other tables. Incorrect entry always flashes and error message.

Numeric Field:

The numeric field can contain only numbers from 0 to 9. An entry of any character flashes an error messages. The individual modules are checked for accuracy and what it has to perform. Each module is subjected to test run along with sample data. The individually tested modules

are integrated into a single system. Testing involves executing the real data information is used in the program the existence of any program defect is inferred from the output. The testing should be planned so that all the requirements are individually tested. A successful test is one that gives out the defects for the inappropriate data and produces an output revealing the errors in the system.

Preparation of Test Data

Taking various kinds of test data does the above testing. Preparation of test data plays a vital role in the system testing. After preparing the test data the system under study is tested using that test data. While testing the system by using test data errors are again uncovered and corrected by using above testing steps and corrections are also noted for future use.

Using Live Test Data:

Live test data are those that are actually extracted from organization files. After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities. Then, the systems person uses this data as a way to partially test the system. In other instances, programmers or analysts extract a set of live data from the files and have them entered themselves.

It is difficult to obtain live data in sufficient amounts to conduct extensive testing. And, although it is realistic data that will show how the system will perform for the typical processing requirement, assuming that the live data entered are in fact typical, such data generally will not test all combinations or formats that can enter the system. This bias toward typical values then does not provide a true systems test and in fact ignores the cases most likely to cause system failure.

Using Artificial Test Data:

Artificial test data are created solely for test purposes, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information systems department, make possible the testing of all login and control paths through the program.

The most effective test programs use artificial test data generated by persons other than those who wrote the programs. Often, an independent team of testers formulates a testing plan, using the systems specifications.

The package “Virtual Private Network” has satisfied all the requirements specified as per software requirement specification and was accepted.

USER TRAINING

Whenever a new system is developed, user training is required to educate them about the working of the system so that it can be put to efficient use by those for whom the system has been primarily designed. For this purpose the normal working of the project was demonstrated to the prospective users. Its working is easily understandable and since the expected users are people who have good knowledge of computers, the use of this system is very easy.

MAINTAINENCE

This covers a wide range of activities including correcting code and design errors. To reduce the need for maintenance in the long run, we have more accurately defined the user’s requirements during the process of system development. Depending on the requirements, this system has been developed to satisfy the needs to the largest possible extent. With development in technology, it may be possible to add many more features based on the requirements in future. The coding and designing is simple and easy to understand which will make maintenance easier.

TESTING STRATEGY :

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

SYSTEM TESTING:

Software once validated must be combined with other system elements (e.g. Hardware, people, database). System testing verifies that all the elements are proper and that overall system function performance is achieved. It also tests to find discrepancies between the system and its original objective, current specifications and system documentation.

UNIT TESTING:

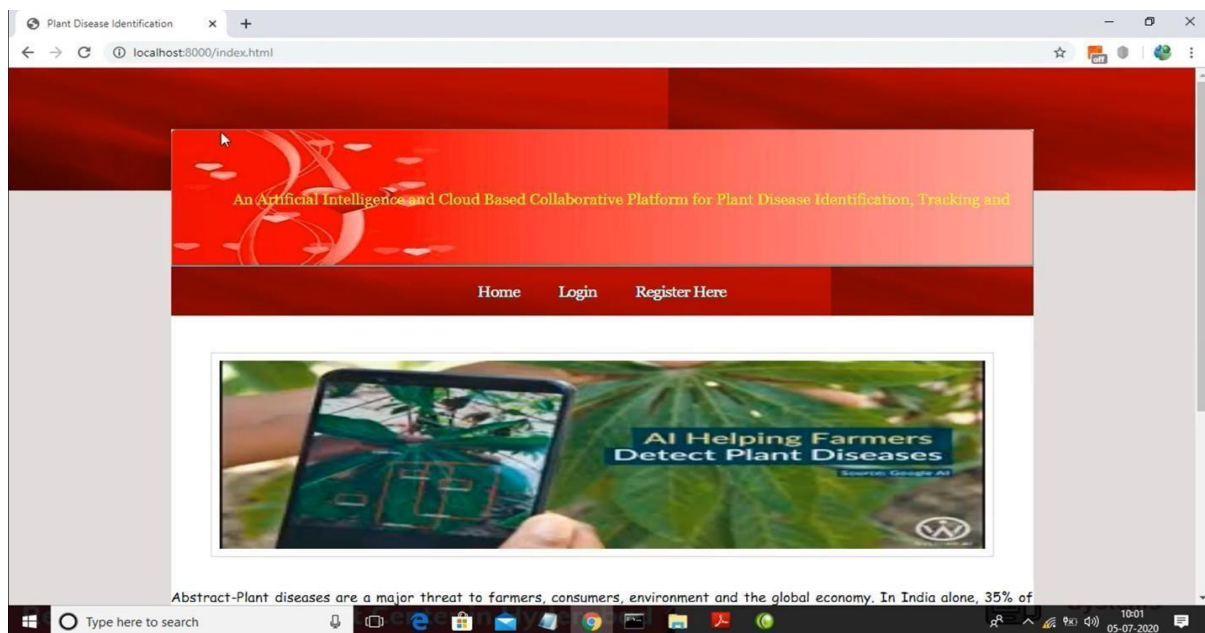
In unit testing different modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goals to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be working satisfactorily as regards to the expected output from the module.

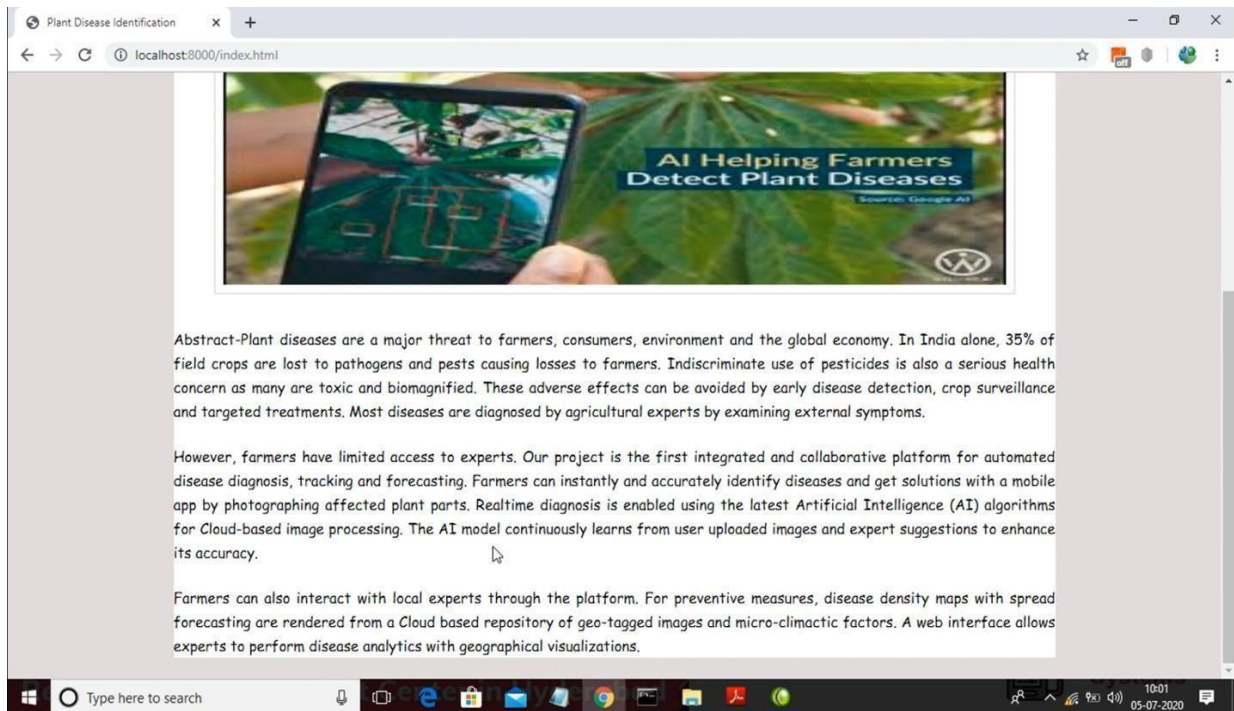
In Due Course, latest technology advancements will be taken into consideration. As part of technical build-up many components of the networking system will be generic in nature so that future projects can either use or interact with this. The future holds a lot to offer to the development and refinement of this project.

CHAPTER-7

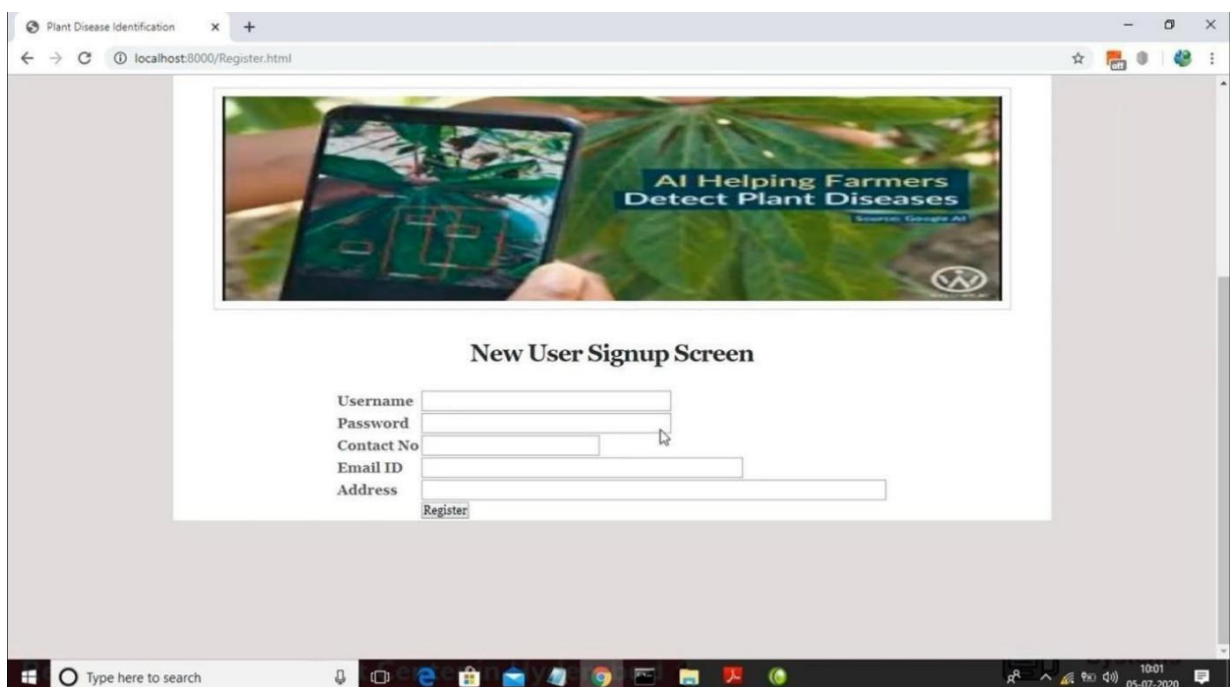
SCREEN SHOTS

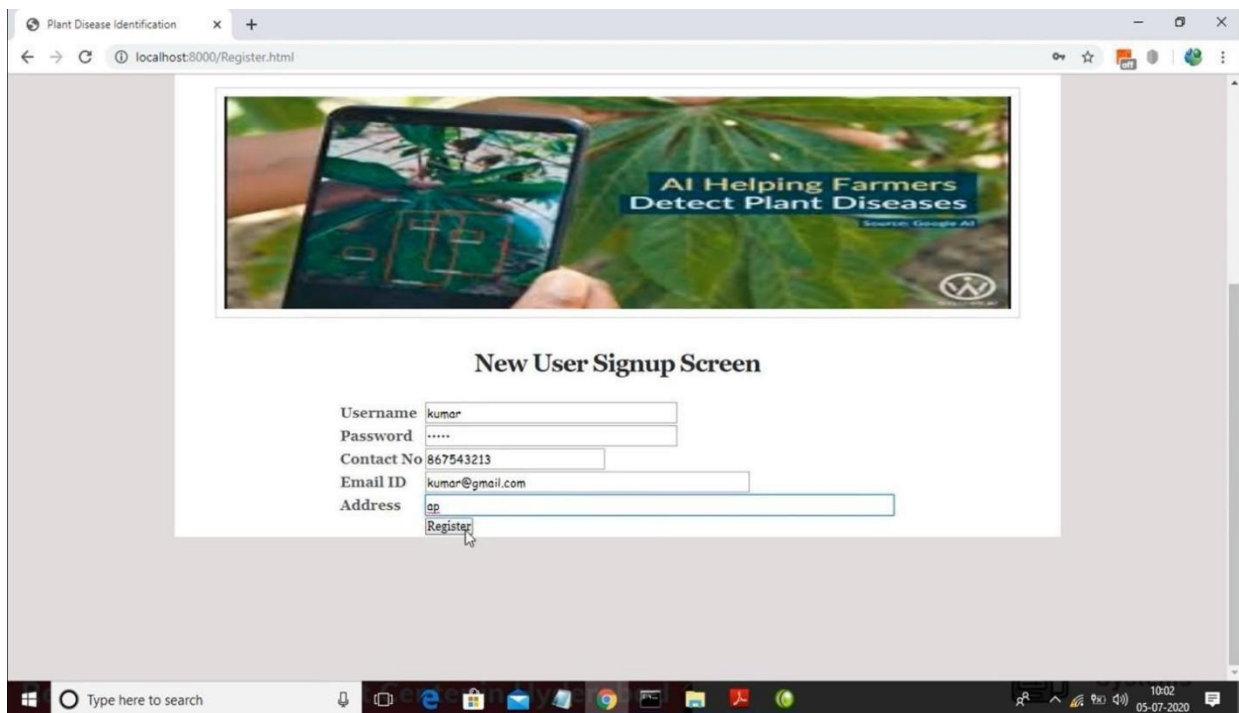
Index.html



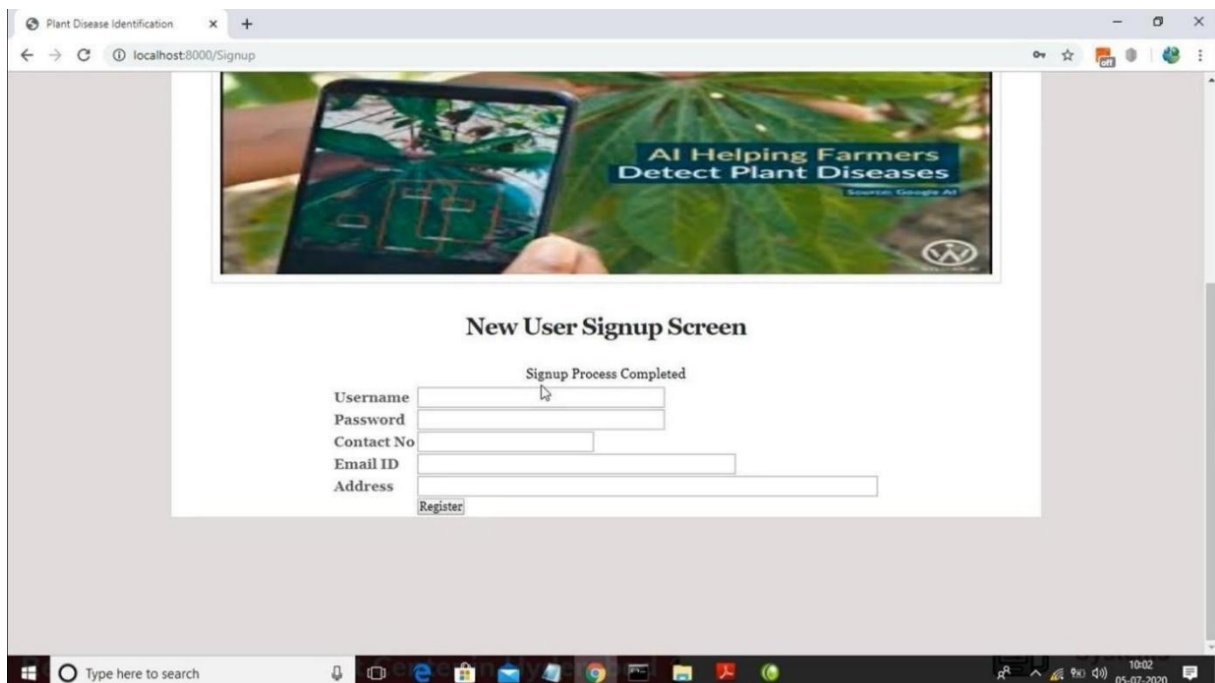


Register.html

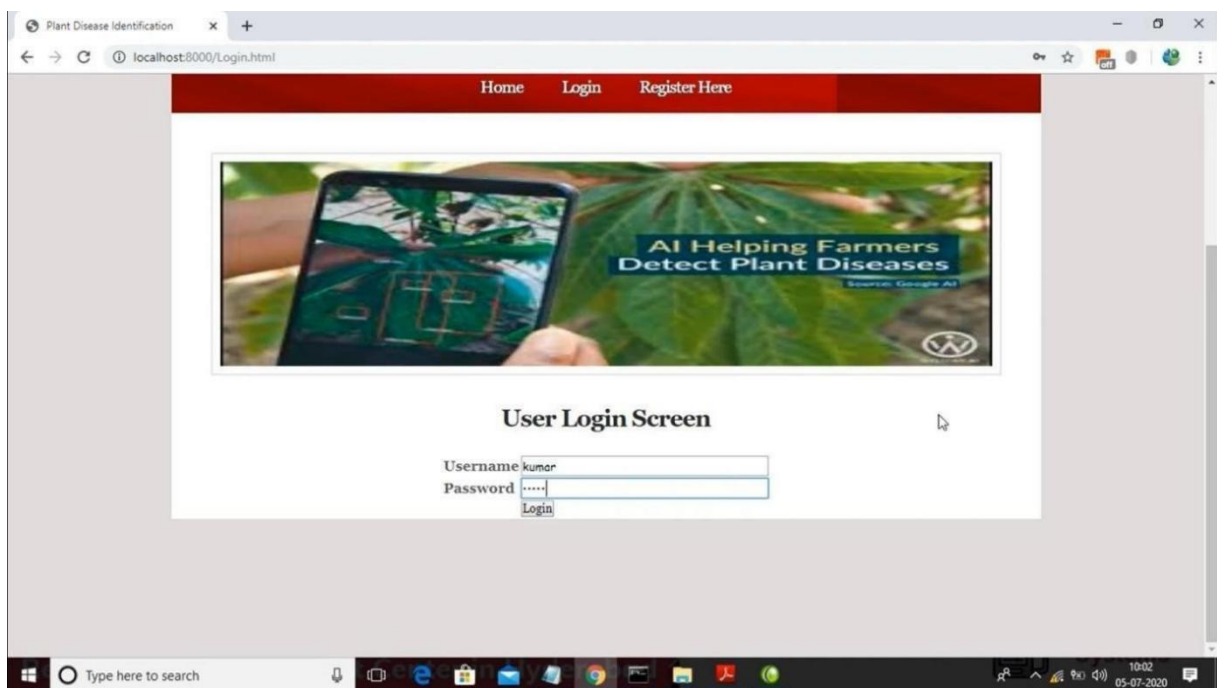
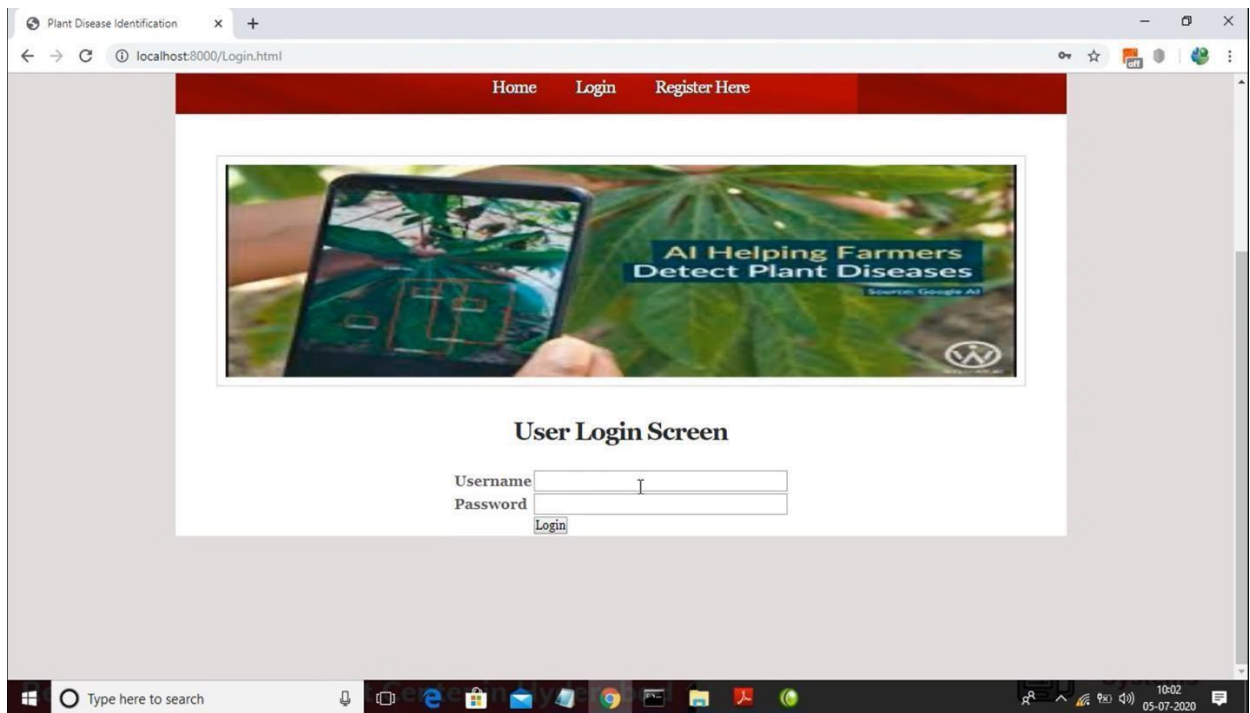




Signup



Login.html



Train CNN Algorithm

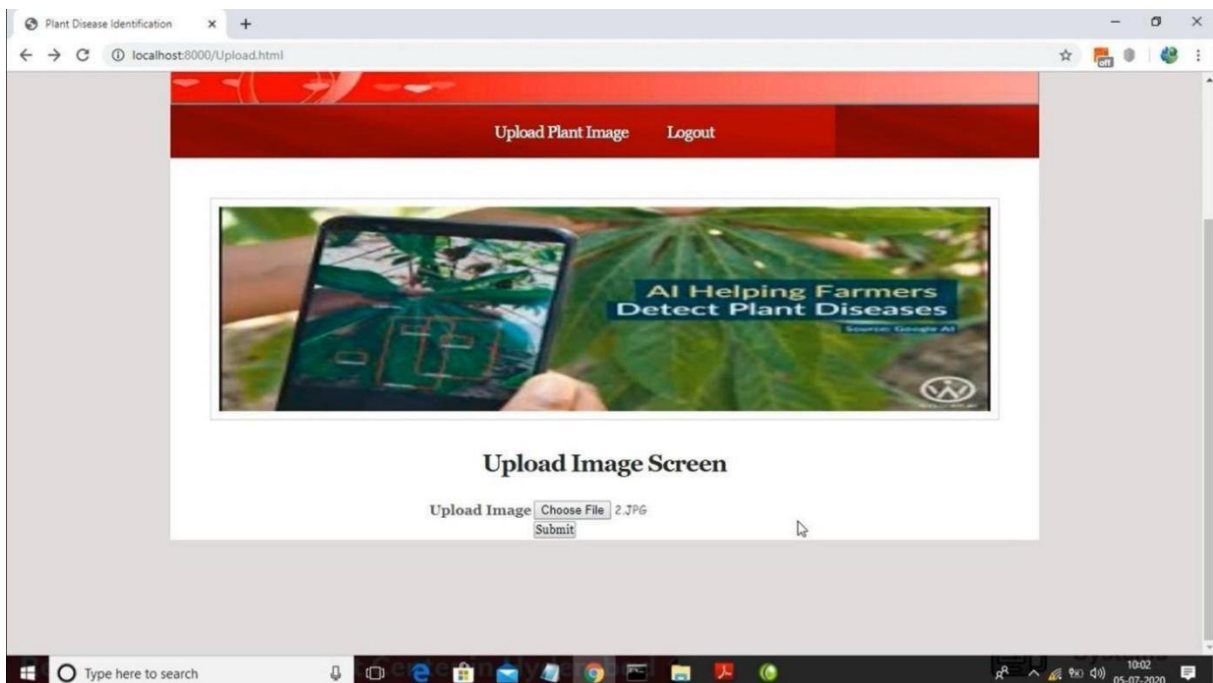
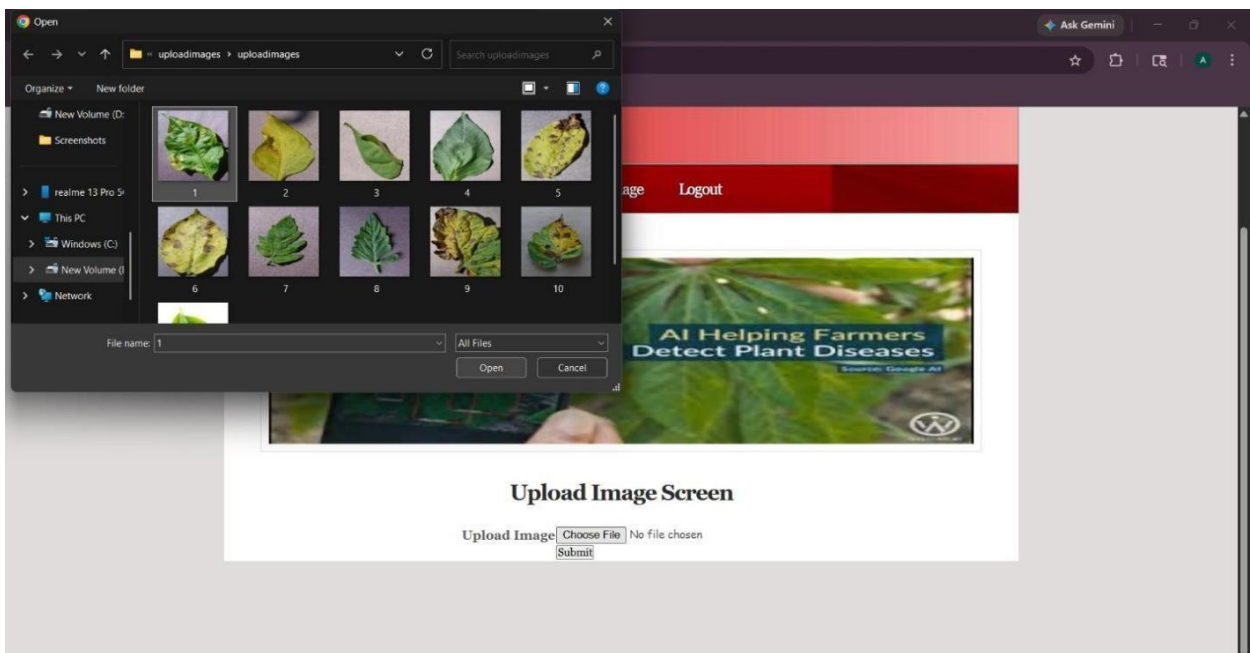
The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/Train". The page has a red header with the text "An Artificial Intelligence and Cloud Based Collaborative Platform for Plant Disease Identification, Tracking and". Below the header is a navigation bar with three buttons: "Train CNN Algorithms", "Upload Plant Image", and "Logout". The main content area features a video player showing a person using a smartphone to identify plant diseases, with the text "AI Helping Farmers Detect Plant Diseases" overlaid. Below the video player is a table with the following data:

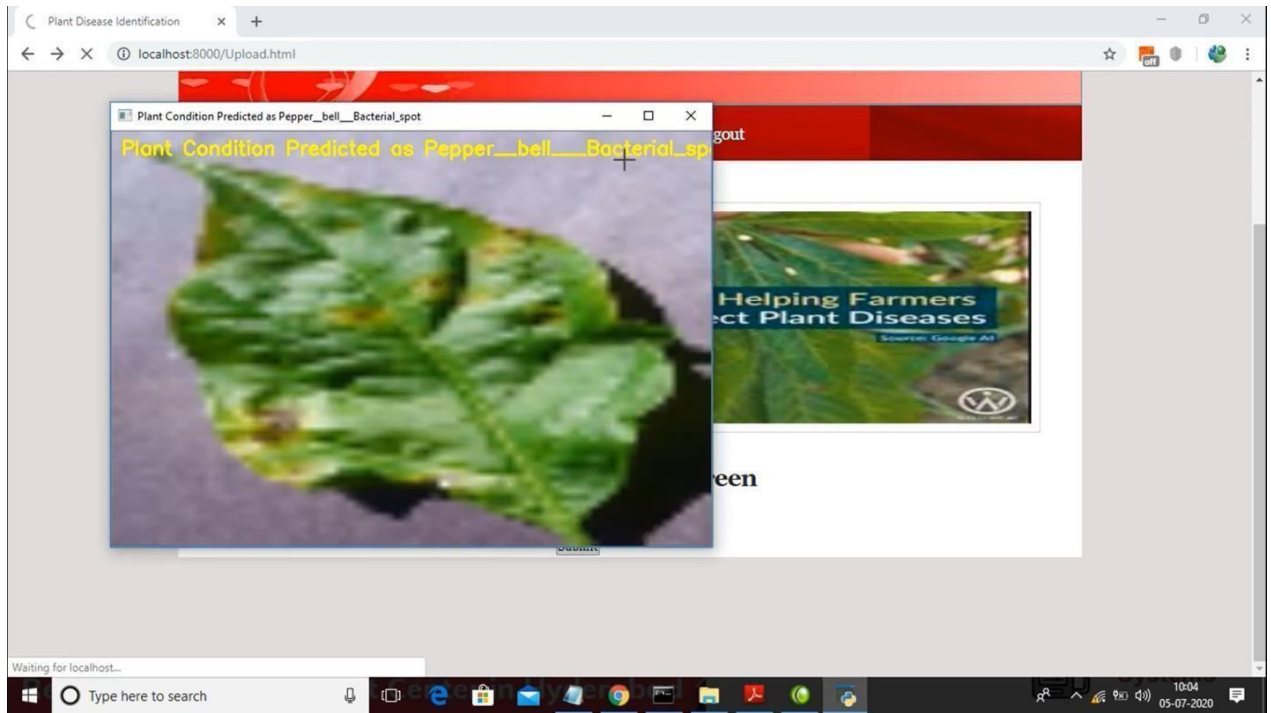
Algorithm Name	Test Accuracy
CNN with Transfer Learning	95.60000002384186
CNN without Transfer Learning	85.6249988079071

Upload.html

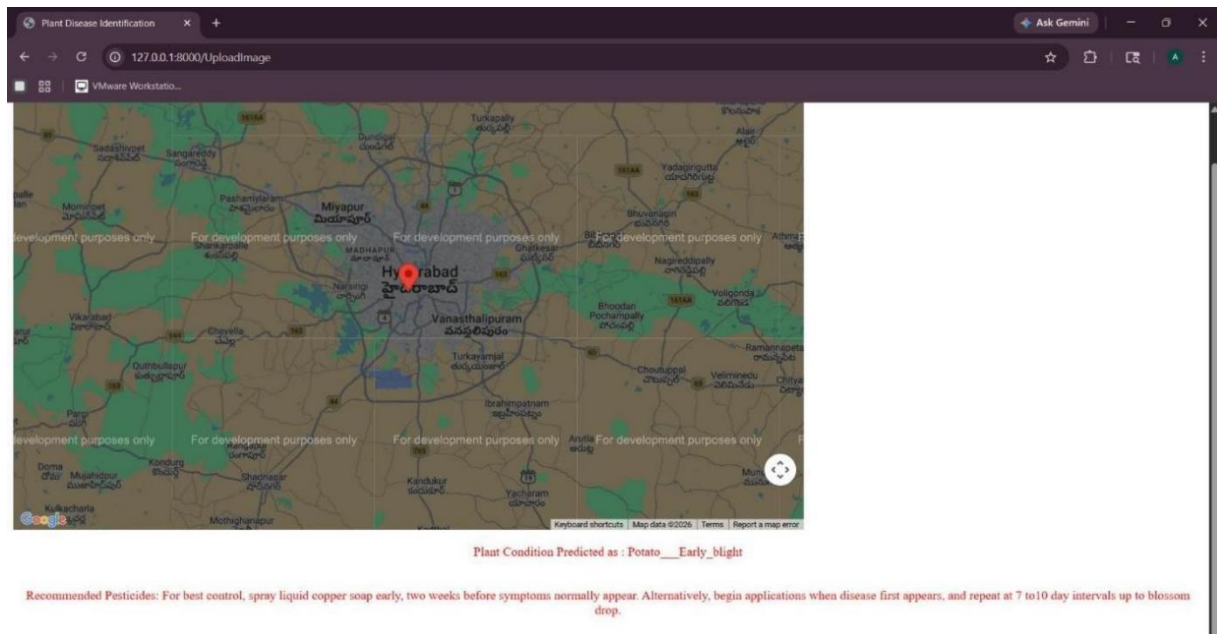
The screenshot shows a web browser window with the address bar displaying "localhost:8000/Upload.html". The page has a red header with the text "Plant Disease Identification". Below the header is a navigation bar with two buttons: "Upload Plant Image" and "Logout". The main content area features a video player showing a person using a smartphone to identify plant diseases, with the text "AI Helping Farmers Detect Plant Diseases" overlaid. Below the video player is a section titled "Upload Image Screen" with a form containing the following elements:

Upload Image: No file chosen





OUTPUT



CHAPTER-8

CONCLUSION

This thesis work presents an automated, low cost and easy to use end-to-end solution to one of the biggest challenges in the agricultural domain for farmers – precise, instant and early diagnosis of crop diseases and knowledge of disease outbreaks - which would be helpful in quick decision making for measures to be adopted for disease control. This proposal innovates a known prior art with the application of deep Convolutional Neural Networks (CNNs) for disease classification, introduction of social collaborative platform for progressively improved accuracy, usage of geocoded images for disease density maps and expert interface for analytics. High performing deep CNN model “Inception” enables real time classification of diseases in the Cloud platform via a user facing mobile app. Collaborative model enables continuous improvement in disease classification accuracy by automatically growing the Cloud based training dataset with user added images for retraining the CNN model. User added images in the Cloud repository also enable rendering of disease density maps based on collective disease classification data and availability of geolocation information within the images. Overall, the results of experiments demonstrate that the proposal has significant potential for practical deployment due to multiple dimensions Cloud based infrastructure is highly scalable and the underlying algorithm works accurately even with large number of disease categories, performs better with high fidelity real-life training data, improves accuracy with increase in the training dataset, is capable of detecting early symptoms of diseases and is able to successfully differentiate between diseases of the same family.

8.1 FUTURE WORK AND EXTENSIONS

Future work involves expanding the model to include more parameters which can improve the correlation to the disease. Research can be extended with the image database, supporting inputs from the farmer on soil, past fertilizer and pesticide treatment along with publicly available environmental factors such as temperature, humidity and rainfall to improve model accuracy and enable disease forecasting. Work can be extended with the number of crop diseases covered and reduce the need for expert intervention except for new types of diseases. For automatic acceptance of user uploaded images into the Training Database for better classification accuracy and least possible human intervention, a simple technique of computing the threshold

based on a mean of all classification scores can be used. Further application of this work could be to support automated time-based monitoring of the disease density maps that can be used to track the progress of a disease and trigger alarms. Predictive analytics can be used to send alerts to the users on the possibility of disease outbreaks near their location.

APPENDIX

A - BIBLIOGRAPHY

- [1] L.Saxena and L. Armstrong, “A survey of image processing techniques for agriculture,” in Proceedings of Asian Federation for Information Technology in Agriculture, 2014, pp. 401-413.
- [2] E. L. Stewart and B. A. McDonald, “Measuring quantitative virulence in the wheat pathogen *Zymoseptoria tritici* using high-throughput automated image analysis,” in *Phytopathology* 104 9, 2014, pp. 985– 992.
- [3] A. Krizhevsky, I. Sutskever and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012.
- [4] TensorFlow.[Online].Available: <https://www.tensorflow.org/>
- [5] D. P. Hughes and and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing,” in *CoRR* abs/1511.08060, 2015.
- [6] S. Raza, G. Prince, J. P. Clarkson and N. M. Rajpoot, “Automatic detection of diseased tomato plants using thermal and stereo visible light images,” in *PLoS ONE*, 2015.
- [7] D. L. Hernández-Rabadán, F. Ramos-Quintana and J. Guerrero Juk, “Integrating soms and a bayesian classifier for segmenting diseased plants in uncontrolled environments,” 2014, in the *Scientific World Journal*, 2014.
- [8] S. Sankaran, A. Mishra, J. M. Maja and R. Ehsani, “Visible-near infrared spectroscopy for detection of huanglongbing in citrus orchards,” in *Computers and Electronics in. Agriculture* 77, 2011, pp. 127–134.
- [9] C. B. Wetterich, R. Kumar, S. Sankaran, J. B. Junior, R. Ehsani and L. G. Marcassa, “A comparative study on application of computer vision and fluorescence imaging spectroscopy for detection of huanglongbing citrus disease in the USA and Brazil,” in *Journal of Spectroscopy*, 2013.

- [10] C. Szegedy, "Rethinking the inception architecture for computer vision," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2818-2826.
- [11] Mango Diseases and Symptoms.[Online].Available
- [12] :[http://vikaspedia.in/agriculture/crop-production/integrated-](http://vikaspedia.in/agriculture/crop-production/integrated-pestmanagement/ipm-for-fruitcrops/ipm-strategies-for-mango/mangodiseases-and-symptoms)
- [13] [pestmanagement/ipm-for- fruitcrops/ipm-strategies-for-](http://vikaspedia.in/agriculture/crop-production/integrated-pestmanagement/ipm-for-fruitcrops/ipm-strategies-for-mango/mangodiseases-and-symptoms)
- [14] [mango/mangodiseases-and-symptoms](http://vikaspedia.in/agriculture/crop-production/integrated-pestmanagement/ipm-for-fruitcrops/ipm-strategies-for-mango/mangodiseases-and-symptoms)
- [15] P. Subrahmanyam, S. Wongkaew, D. V. R. Reddy, J. W. Demski, D. McDonald, S. B.Sharma and D. H. Smith, "Field Diagnosis of Groundnut Diseases". Monograph. International Crops Research Institute for the Semi-Arid Tropics, 1992.
- [16] K. Elangovan and S. Nalini, "Plant disease classification using image segmentation andSVM techniques", International Journal of Computational Intelligence Research, vol. 13(7), pp. 1821-1828, 2017.
- [17] Dhingra, G., Kumar, V. & Joshi, H.D. Study of digital image processing techniques for leaf disease detection and classification. Multimed Tools Appl 77, 19951–20000 (2018). <https://doi.org/10.1007/s11042-017-5445-8>
- [18] M. M. Ozguven, K. Adem, Automatic detection and classification of leaf spot disease in sugar beet using deep learning algorithms, Physica A: Statistical Mechanic sand its Applications, Volume535,2019, <https://doi.org/10.1016/j.physa.2019.122537>.
- [19] Wang, H.; Shang, S.; Wang, D.; He, X.; Feng, K.; Zhu, H. Plant Disease Detection and Classification Method Based on the Optimized Lightweight YOLOv5 Model. Agriculture 2022, 12, 931. <https://doi.org/10.3390/agriculture12070931>.
- [20] Rakesh Kaundal, Amar S Kapoor and Gajendra PS Raghava "Machine learning technique in disease forecasting: a case study on rice blast prediction," 2006.
- [21] Srdjan BMC Bioinformatics, Sladojevic, Marko Arsenovic, Andras Anderla, Dubravko Culibrk, and Darko Stefanovic, Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification, Hindawi Publishing

Corporation, Computational Intelligence and Neuroscience Volume 2016, Article ID 3289801, 11 pages <http://dx.doi.org/10.1155/2016/328980>

- [22] J.Howse,OpenCV ComputerVision with Python, PacktPublishing, Birmingham, UK, 2013.
- [23] Thomas, S.; Kuska, M.T.; Bohnenkamp, D.; Brugger, A.; Alisaac, E.; Wahabzada, M.; Behmann, J.; Mahlein, A.K. Benefits of hyperspectral imaging for plant disease detection and plant protection: A technical perspective. J. Plant Dis. Prot. 2018, 125, 5–20.
- [24] Moamen,A.A.; Jamali, N. Opportunistic sharing of continuous mobile sensing data for energy and power conservation. IEEE Trans. Serv. Comput. 2020, 13, 503–514.

B - TECHNOLOGY USED

Python Introduction

Python is a general purpose, dynamic, high level and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.

Python is easy to learn yet powerful and versatile scripting language which makes it attractive for Application Development.

Python's syntax and dynamic typing with its interpreted nature, makes it an ideal language for scripting and rapid application development.

Python supports multiple programming pattern, including object oriented, imperative and functional or procedural programming styles.

Python is not intended to work on special area such as web programming. That is why it is known as multipurpose because it can be used with web, enterprise, 3D CAD etc.

We don't need to use data types to declare variable because it is dynamically typed so we can write `a=10` to assign an integer value in an integer variable.

Python makes the development and debugging fast because there is no compilation step included in python development and edit-test-debug cycle is very fast.

Python History

- Python laid its foundation in the late 1980s.
- The implementation of Python was started in the December 1989 by **Guido Van Rossum** at CWI in Netherland.
- In February 1991, van Rossum published the code (labeled version 0.9.0) to alt.sources.
- In 1994, Python 1.0 was released with new features like: lambda, map, filter, and reduce.
- Python 2.0 added new features like: list comprehensions, garbage collection system.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify fundamental flaw of the language.

- ABC programming language is said to be the predecessor of Python language which was capable of Exception Handling and interfacing with Amoeba Operating System.
- Python is influenced by following programming languages:
- ABC language. ○ Modula-3

Python Features

Python provides lots of features that are listed below.

1) Easy to Learn and Use

Python is easy to learn and use. It is developer-friendly and high level programming language.

2) Expressive Language

Python language is more expressive means that it is more understandable and readable.

3) Interpreted Language

Python is an interpreted language i.e. interpreter executes the code line by line at a time. This makes debugging easy and thus suitable for beginners.

4) Cross-platform Language

Python can run equally on different platforms such as Windows, Linux, Unix and Macintosh etc. So, we can say that Python is a portable language.

5) Free and Open Source

Python language is freely available at official web address. The source-code is also available. Therefore it is open source.

6) Object-Oriented Language

Python supports object oriented language and concepts of classes and objects come into existence.

7) Extensible

It implies that other languages such as C/C++ can be used to compile the code and thus it can be used further in python code.

8) Large Standard Library

Python has a large and broad library and provides rich set of module and functions for rapid application development.

9) GUI Programming Support

Graphical user interfaces can be developed using Python.

10) Integrated

It can be easily integrated with languages like C, C++, JAVA etc.

Python Applications

Python is known for its general purpose nature that makes it applicable in almost each domain of software development. Python as a whole can be used in any sphere of development.

Here, the following python applications were applied to run the project.

1) Web Applications

We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML and XML, JSON, Email processing, request, BeautifulSoup, Feedparser etc. It also provides Frameworks such as Django, Pyramid, Flask etc to design and develop web based applications. Some important developments are: PythonWikiEngines, Pycoco, PythonBlogSoftware etc.

2) Desktop GUI Applications

Python provides Tk GUI library to develop user interface in python based application. Some other useful toolkits wxWidgets, Kivy, pyqt that are useable on several platforms. The Kivy is popular for writing multitouch applications.

3) Software Development

Python is helpful for software development process. It works as a support language and can be used for build control and management, testing etc.

4) Scientific and Numeric

Python is popular and widely used in scientific and numeric computing. Some useful library and package are SciPy, Pandas, IPython etc. SciPy is group of packages of engineering, science and mathematics.

5) Business Applications

Python is used to build Bussiness applications like ERP and e-commerce systems. Tryton is a high level application platform.

6) Console Based Application

We can use Python to develop console based applications. For example: **IPython**.

7) Audio or Video based Applications

Python is awesome to perform multiple tasks and can be used to develop multimedia applications. Some of real applications are: TimPlayer, cplay etc.

8) 3D CAD Applications

To create CAD application Fandango is a real application which provides full features of CAD.

9) Enterprise Applications

Python can be used to create applications which can be used within an Enterprise or an Organization. Some real time applications are: OpenErp, Tryton, Picalo etc.

10) Applications for Images

Using Python several application can be developed for image. Applications developed are: VPython, Gogh, imgSeek etc.

There are several such applications which can be developed using Python

How to Install Python (Environment Set-up)

In this section of the tutorial, we will discuss the installation of python on various operating systems.

Why Python

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-orientated way or a functional way.

Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial. However, Python 2, although not being updated with anything other than security updates, is still quite popular.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Installation on Windows

Visit the link <https://www.python.org/downloads/> to download the latest release of Python. In this process, we will install Python 3.6.7 on our Windows operating system.

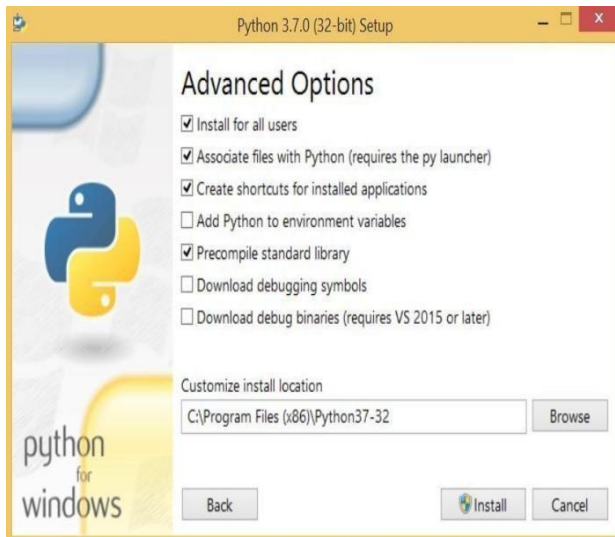
Double-click the executable file which is downloaded; the following window will open. Select Customize installation and proceed.



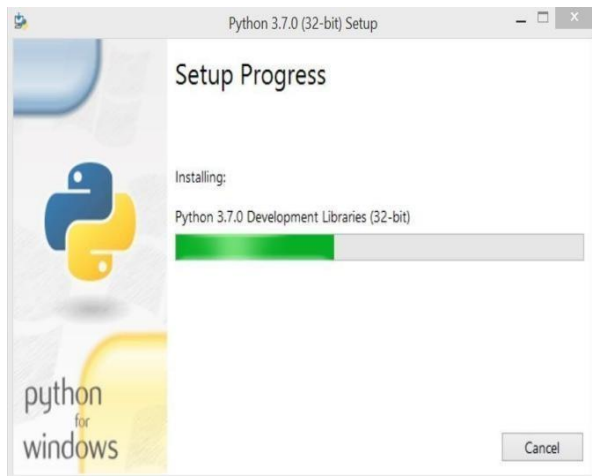
The following window shows all the optional features. All the features need to be installed and are checked by default; click next to continue.



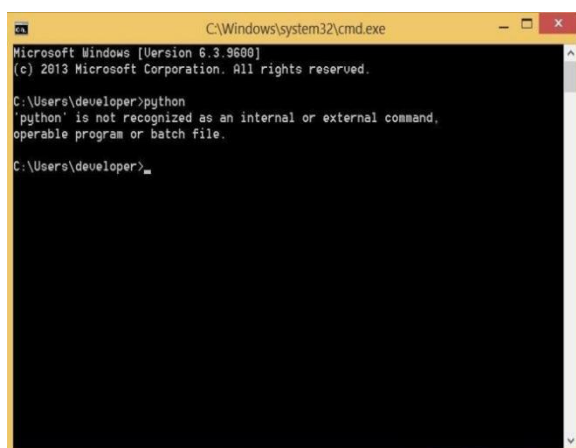
The following window shows a list of advanced options. Check all the options which you want to install and click next. Here, note that the first check-box (install for all users) must be checked.



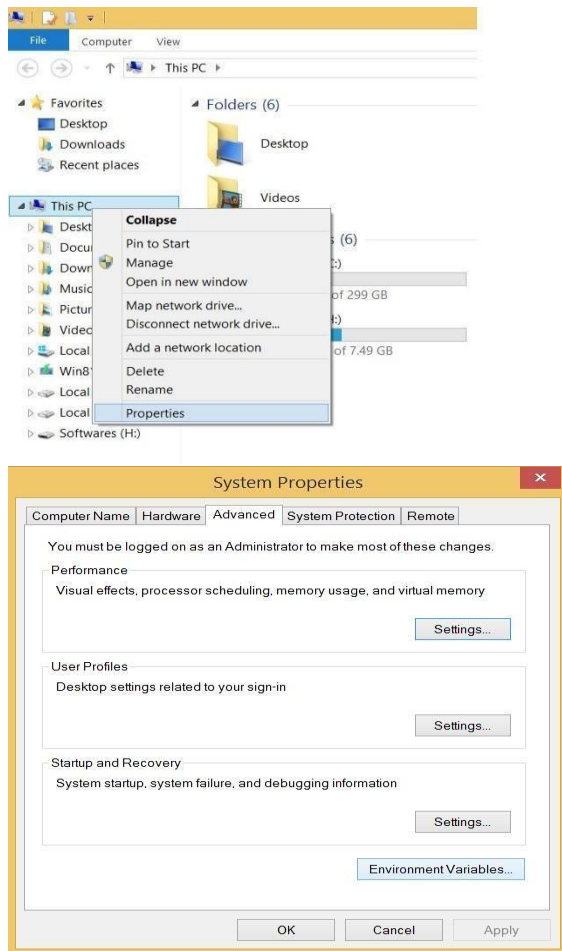
Now, we are ready to install python-3.6.6. Lets install it.



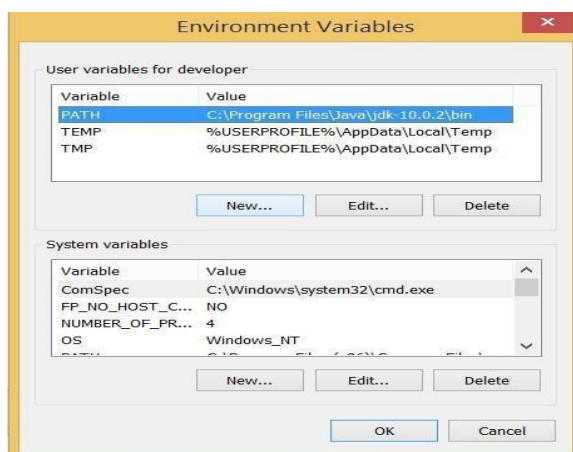
Now, try to run python on the command prompt. Type the command python in case of python2 or python3 in case of python3. It will show an error as given in the below image. It is because a path is not set.



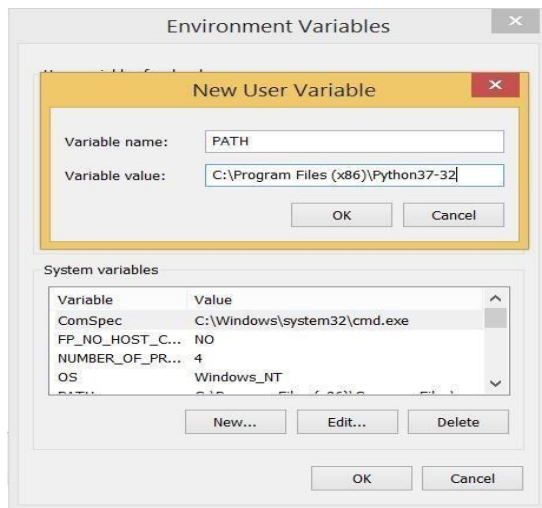
To set the path of python, the right click on "my computer" and go to Properties → Advanced → Environment Variables.



Add the new path variable in the user variable section.



Type PATH as the variable name and set the path to the installation directory of the python shown in the below image.



Now, the path is set, we are ready to run python on our local system. Restart CMD, and type python again. It will open the python interpreter shell where we can execute the python statements.

Virtual Environments and Packages

Introduction

Python applications will often use packages and modules that don't come as part of the standard library. Applications will sometimes need a specific version of a library, because the application may require that a particular bug has been fixed or the application may be written using an obsolete version of the library's interface.

This means it may not be possible for one Python installation to meet the requirements of every application. If application A needs version 1.0 of a particular module but application B needs version 2.0, then the requirements are in conflict and installing either version 1.0 or 2.0 will leave one application unable to run.

The solution for this problem is to create a virtual environment, a self-contained directory tree that contains a Python installation for a particular version of Python, plus a number of additional packages.

Different applications can then use different virtual environments. To resolve the earlier example of conflicting requirements, application A can have its own virtual environment with version 1.0 installed while application B has another virtual environment with version 2.0. If

application B requires a library be upgraded to version 3.0, this will not affect application A's environment.

Creating Virtual Environments

The module used to create and manage virtual environments is called `venv`. `venv` will usually install the most recent version of Python. If multiple versions of python available then select a specific Python version by running `python3` or whichever version is needed.

To create a virtual environment, decide upon a directory where it is to be placed, and run the `venv` module as a script with the directory path:

```
python3 -m venv tutorial-env
```

This will create the `tutorial-env` directory if it doesn't exist, and also create directories inside it containing a copy of the Python interpreter, the standard library, and various supporting files.

A common directory location for a virtual environment is `.venv`. This name keeps the directory typically hidden in shell and thus out of the way while giving it a name that explains why the directory exists. It also prevents clashing with `.env` environment variable definition files that some tooling supports.

Once a virtual environment is created, activate it.

On Windows, run:

```
tutorial-env\Scripts\activate.bat
```

Unix or MacOS, run:

```
source tutorial-env/bin/activate
```

(This script is written for the bash shell. If you use the `csh` or `fish` shells, there are alternate `activate.csh` and `activate.fish` scripts you should use instead.)

Activating the virtual environment will change shell's prompt to show what virtual environment, and modify the environment so that running `python` will get that particular version and installation of Python. For example:

```
$ source ~/envs/tutorial-env/bin/activate
```

```
(tutorial-env) $ python
```

```
Python 3.5.1 (default, May 6 2016, 10:59:36)
```

```
...
```

```
>>> import sys
```

```
>>> sys.path
```

```
['', '/usr/local/lib/python35.zip', ...,
```

```
'~/envs/tutorial-env/lib/python3.5/site-packages']
```

```
>>>
```

Managing Packages with pip

All can install, upgrade, and remove packages using a program called pip. By default pip will install packages from the Python Package Index, <<https://pypi.org>>. All can browse the Python Package Index by going to it in reference web browser, or can use pip's limited search feature:

```
(tutorial-env) $ pip search astronomy skyfield - Elegant
```

```
astronomy for Python gary - Galactic astronomy and  
gravitational dynamics.
```

```
novas- The United States Naval Observatory NOVAS astronomy library
```

```
astroob- Provides astronomy ephemeris to plan telescope observations
```

```
PyAstronomy- A collection of astronomy related tools for Python.
```

```
...
```

pip has a number of subcommands: “search”, “install”, “uninstall”, “freeze”, etc. (Consult the Installing Python Modules guide for complete documentation for pip.)

User can install the latest version of a package by specifying a package's name:

```
(tutorial-env) $ pip install novas
```

```
Collecting novas
```

```
Downloading novas-3.1.1.3.tar.gz (136kB)
```

```
Installing collected packages: novas
```

Running setup.py install for novas

Successfully installed novas-3.1.1.3

User can also install a specific version of a package by giving the package name followed by == and the version number:

```
(tutorial-env) $ pip install requests==2.6.0
```

Collecting requests==2.6.0

Using cached requests-2.6.0-py2.py3-none-any.whl

Installing collected packages: requests

Successfully installed requests-2.6.0

After running this command, pip will notice that the requested version is already installed and do nothing. a different version number can be supplied to get that version, or can run pip install --upgrade to upgrade the package to the latest version:

```
(tutorial-env) $ pip install --upgrade requests
```

Collecting requests

Installing collected packages: requests Found

existing installation: requests 2.6.0

Uninstalling requests-2.6.0:

Successfully uninstalled requests-2.6.0 Successfully installed requests-2.7.0 pip uninstall followed by one or more package names will remove the packages from the virtual environment. pip show will display information about a particular package:

```
(tutorial-env) $ pip show requests
```

Metadata-Version: 2.0

Name: requests

Version: 2.7.0

Summary: Python HTTP for Humans.

Home-page: <http://python-requests.org>

Author: Kenneth Reitz

Author-email: me@kennethreitz.com

License: Apache 2.0

Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages Requires:

pip list will display all of the packages installed in the virtual environment:

```
(tutorial-env) $ pip list
```

novas (3.1.1.3) numpy

```
(1.9.2)  pip    (7.0.3)
requests      (2.7.0)
setuptools (16.0)
```

pip freeze will produce a similar list of the installed packages, but the output uses the format that pip install expects. A common convention is to put this list in a requirements.txt file:

```
(tutorial-env) $ pip freeze > requirements.txt
```

```
(tutorial-env) $ cat requirements.txt
```

```
novas==3.1.1.3 numpy==1.9.2
```

```
requests==2.7.0
```

The requirements.txt can then be committed to version control and shipped as part of an application. Users can then install all the necessary packages with install -r:

```
(tutorial-env) $ pip install -r requirements.txt
```

```
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
```

```
...
```

```
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
```

```
...
```

```
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
```

```
...
```

```
Installing collected packages: novas, numpy, requests
```

```
Running setup.py install for novas
```

```
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0 pip has many more options.
```

Consult the Installing Python Modules guide for complete documentation for pip. When a package is written and want to make it available on the Python Package Index, consult the Distributing Python Modules guide.